

Árboles N-arios: Trie

Ejercicio 2

Sabiendo que el orden de complejidad para el peor caso de la operación `search()` es de $O(m |\Sigma|)$. Proponga una versión de la operación `search()` cuya complejidad sea $O(m)$.

El orden de complejidad para el peor caso de la operación `search()` en los Tries es $O(m |\Sigma|)$ porque, de tener implementado el atributo `children` del nodo raíz como una `LinkedList`, puede que se requiera recorrer la lista completa hasta la última posición si las palabras a buscar no comparten ningún otro inicio común.

Para solucionarlo, podemos implementar un Trie cuyos nodos tengan su atributo “children” implementado mediante arrays, de manera tal que el direccionamiento sea directo para todos los elementos pertenecientes al árbol. De esta forma evitamos el peor caso donde se recorre el alfabeto entero.

No obstante, una implementación de Trie utilizando arrays presenta el problema de que su complejidad espacial crece demasiado, dado que por cada letra del alfabeto vamos a tener otro array de longitud completa del alfabeto, y así sucesivamente.

Ejercicio 5

Implementar un algoritmo que dado los Trie T_1 y T_2 devuelva `True` si estos pertenecen al mismo documento y `False` en caso contrario. Se considera que un Trie pertenece al mismo documento cuando:

- 1. Ambos Trie sean iguales (esto se debe cumplir)**
- 2. ~~El Trie T_1 contiene un subconjunto de las palabras del Trie T_2~~**
- 3. Si la implementación está basada en `LinkedList`, considerar el caso donde las palabras hayan sido insertadas en un orden diferente.**

En otras palabras, analizar si todas las palabras de T_1 se encuentran en T_2 .

Analizar el costo computacional.

El código de la solución se encuentra subido al repositorio correspondiente.

La solución propuesta presenta una complejidad temporal de $O(n_1 + m \cdot n_2)$, puesto que el costo computacional se divide en:

- Recorrer los n_1 nodos del primer árbol T_1 de manera recursiva para formar las palabras tiene un costo de $O(n_1)$;
- Buscar las M palabras en el árbol T_2 , recorriendo sus n_2 nodos, tiene un costo computacional de $O(m \cdot n_2)$.